

REPUBLIQUE DU CAMEROUN

Paix – Travail – Patrie

MINISTERE DE L'ENSEIGNEMENT
SUPERIEUR

UNIVERSITE DE YAOUNDE I

REPUBLIC OF CAMEROON

Peace – Work – FatherLand

MINISTRY OF HIGHER EDUCATION

UNIVERSITY OF YAOUNDE I



INSTITUT UNIVERSITAIRE SAINT JEAN
SAINT JEAN INGÉNIEUR

Cahier de conception pour le projet du cours de sécurité cloud.

Filière : Sécurité des Systèmes d'Information

Thème :

Rédigé par :

ANABA NGOUMOU Rachel
&
HARVIS LANDRY Fotseu Fonkwa

Sous l'encadrement

Académique de :

M. NGUIMBUS Emmanuel

Enseignant à l'Institut
Universitaire Saint Jean

I.Contexte du projet

L'intégration de Barbican au sein de notre infrastructure répond à la nécessité critique de centraliser et de sécuriser la gestion des secrets. Toutefois, l'analyse de l'architecture actuelle a mis en exergue plusieurs limites opérationnelles et techniques qui compromettent la fluidité des déploiements et la continuité de service. Ces dysfonctionnements, identifiés comme des freins majeurs à l'efficacité du système, s'articulent principalement autour de trois axes : le déficit de communication sur l'état des ressources, la complexité des protocoles d'accès pour les entités externes et les risques inhérents à la gestion manuelle des cycles de vie des actifs de sécurité.

1.Optimisation du cycle de vie et alertes proactives

Le premier enjeu majeur de ce projet réside dans le manque de visibilité sur l'état des secrets. Actuellement, l'absence de système de notification automatisé crée une opacité critique car lorsqu'une clé API est modifiée ou qu'un certificat approche de sa fin de validité, ni les utilisateurs responsables, ni les applications dépendantes ne sont informés. Cette lacune entraîne inévitablement des erreurs opérationnelles imprévisibles, car la détection des problèmes ne se fait qu'au moment de la rupture du service, empêchant toute maintenance préventive.

2.Standardisation et sécurisation de l'accès pour les tiers

Un obstacle structurel freine l'adoption globale de la solution, il s'agit de la rigidité du système d'authentification actuel. Nativement, Barbican est étroitement lié à l'écosystème OpenStack, limitant ainsi l'accès aux seuls comptes utilisateurs ou services internes à cette plateforme. Cette complexité d'authentification exclut les applications tierces externes qui ne possèdent pas d'identité au sein d'OpenStack. L'objectif est donc de lever ces barrières pour permettre à des services hétérogènes de consommer des secrets de manière sécurisée et fluide, sans dépendance exclusive à l'infrastructure d'origine.

3.Automatisation de la rotation pour la continuité de service

Enfin, la gestion actuelle des éléments de sécurité repose sur des processus manuels de renouvellement des clés API et des certificats SSL/TLS. Cette méthode est source d'une vulnérabilité importante car l'oubli ou le retard d'un renouvellement humain provoque systématiquement des interruptions brutales des services critiques. En intégrant Barbican pour automatiser ces cycles de rotation, le projet vise à garantir une haute disponibilité du système, éliminant les temps d'arrêt causés par l'expiration accidentelle des certificats et renforçant la posture de sécurité globale par une rotation plus fréquente et fiable.

II.Présentation des objectifs

Pour pallier les lacunes de l'infrastructure actuelle et garantir une gestion des secrets à la fois agile et résiliente, notre projet s'articule autour d'objectifs stratégiques majeurs. Il ne s'agit plus

seulement de stocker des données sensibles, mais de transformer la sécurité en un service fluide et transparent. Cette ambition se traduit par la mise en place d'une architecture ouverte aux systèmes externes, l'éradication des interventions manuelles critiques et l'instauration d'une visibilité totale sur l'état de conformité de nos ressources. Ces axes de développement visent à transformer nos contraintes actuelles en un socle technologique robuste et automatisé.

1.Standardisation et ouverture de l'authentification

L'objectif prioritaire est de définir un protocole d'accès universel permettant de standardiser l'authentification des applications tierces. Il s'agit de s'affranchir des limitations natives liées à l'écosystème OpenStack pour offrir une méthode d'interconnexion sécurisée, capable d'intégrer des services externes. En simplifiant ces mécanismes d'entrée, nous garantissons que chaque composant du système, quelle que soit son origine, puisse consommer les secrets nécessaires de manière homogène.

2.Automatisation du cycle de vie des secrets

Afin de supprimer les risques de rupture de service, le projet vise à automatiser intégralement le renouvellement des certificats SSL/TLS et des clés API. En remplaçant les interventions manuelles par des flux de travail programmés, nous assurons une rotation fluide et systématique des éléments de sécurité. Cette automatisation garantit que les actifs sont renouvelés bien avant leur expiration, assurant ainsi une disponibilité continue des infrastructures critiques sans nécessiter d'actions humaines répétitives.

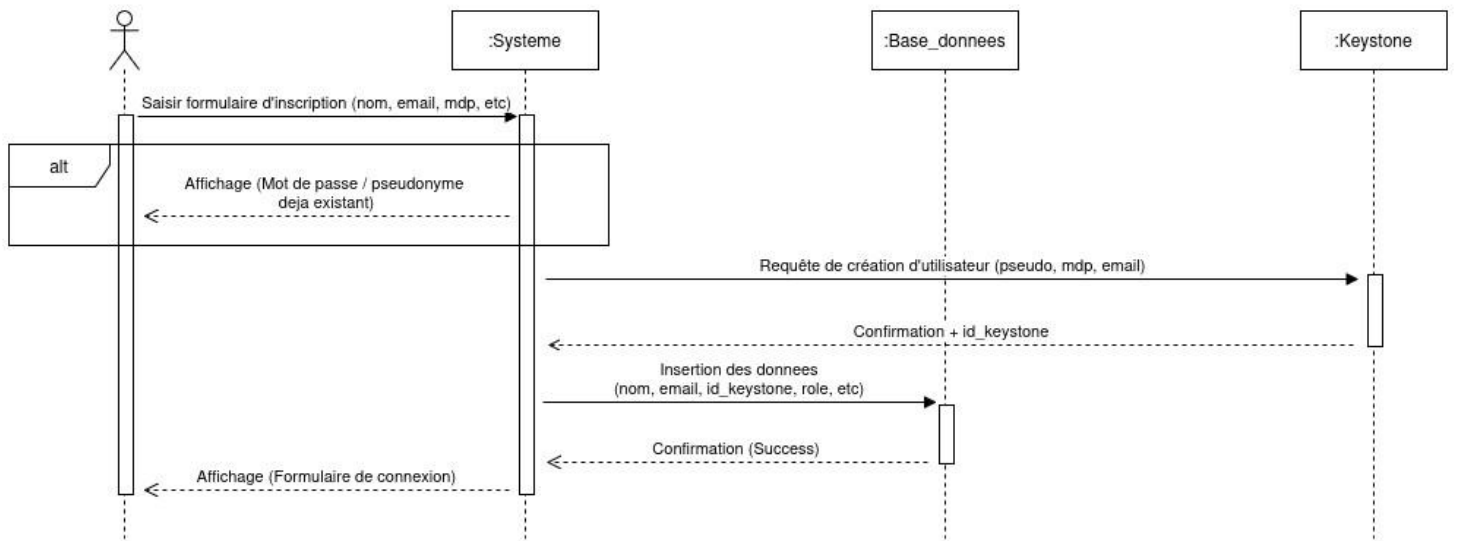
3.Monitoring global et observabilité de la conformité

Enfin, le projet déploie une couche de supervision dédiée à la surveillance active des applications et de la validité de leurs secrets. Ce dispositif de monitoring permettra de suivre en temps réel l'état de santé des clés API et la date d'échéance des certificats. Grâce à la mise en place d'alertes proactives, les utilisateurs finaux disposeront d'une visibilité complète sur le parc de secrets, permettant de détecter instantanément toute anomalie ou défaillance.

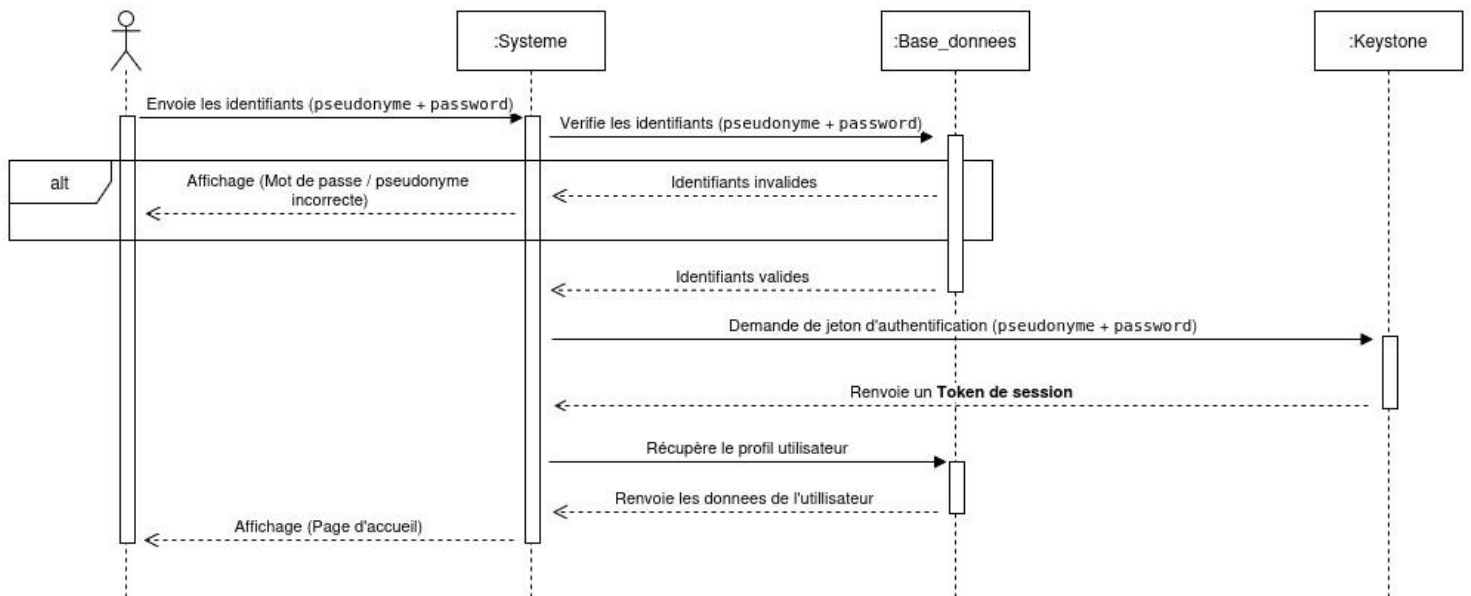
CONCEPTION FONCTIONNELLE

III. Diagrammes de séquence

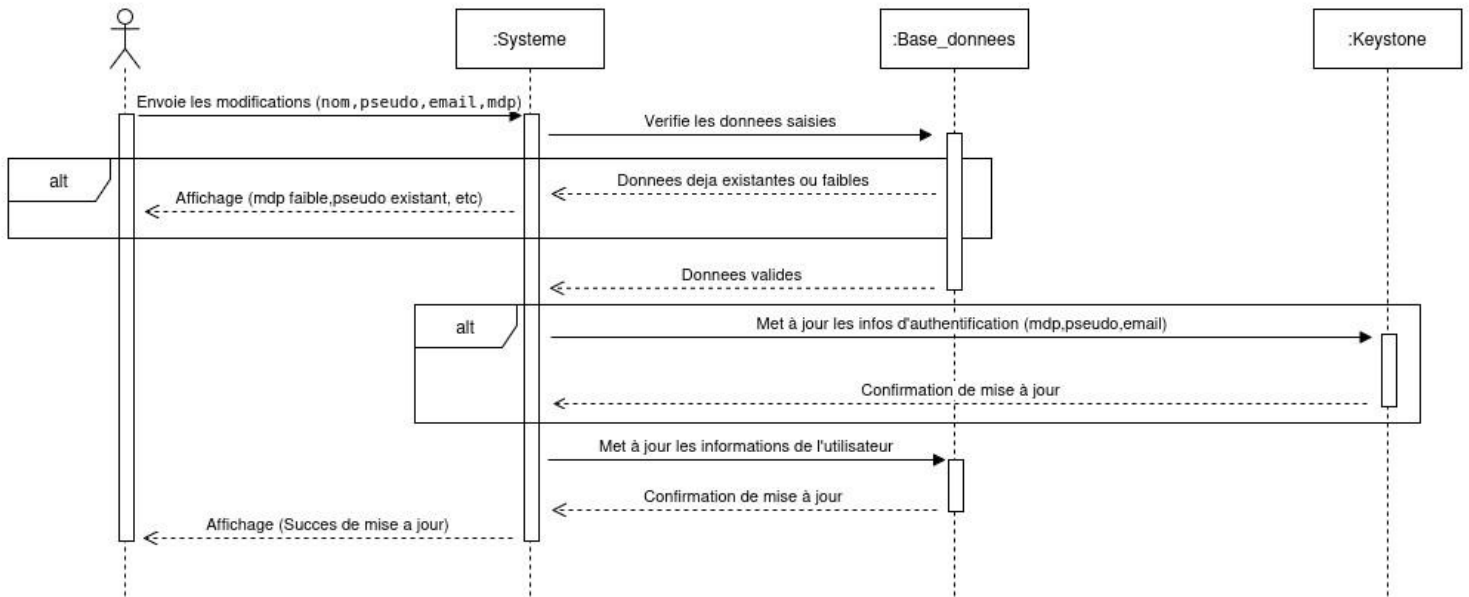
1. Créer un compte



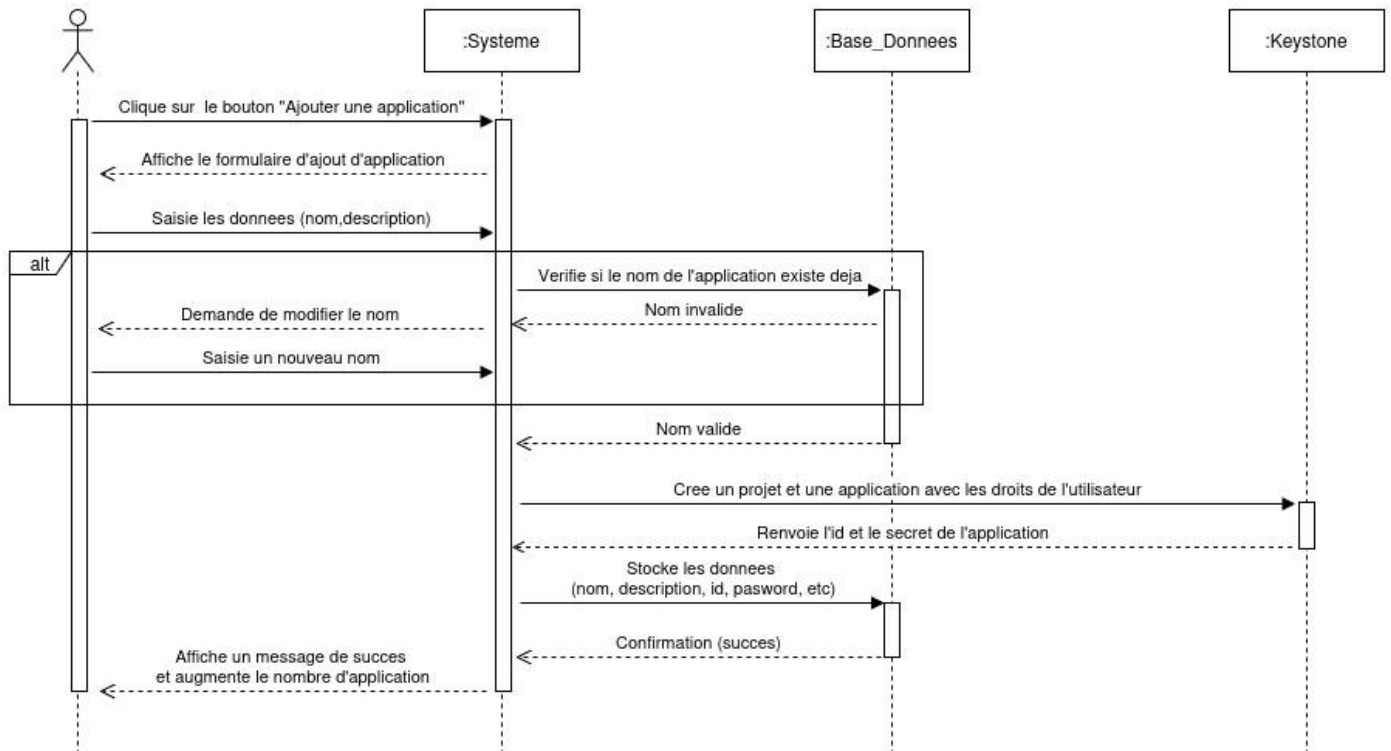
2. Se connecter



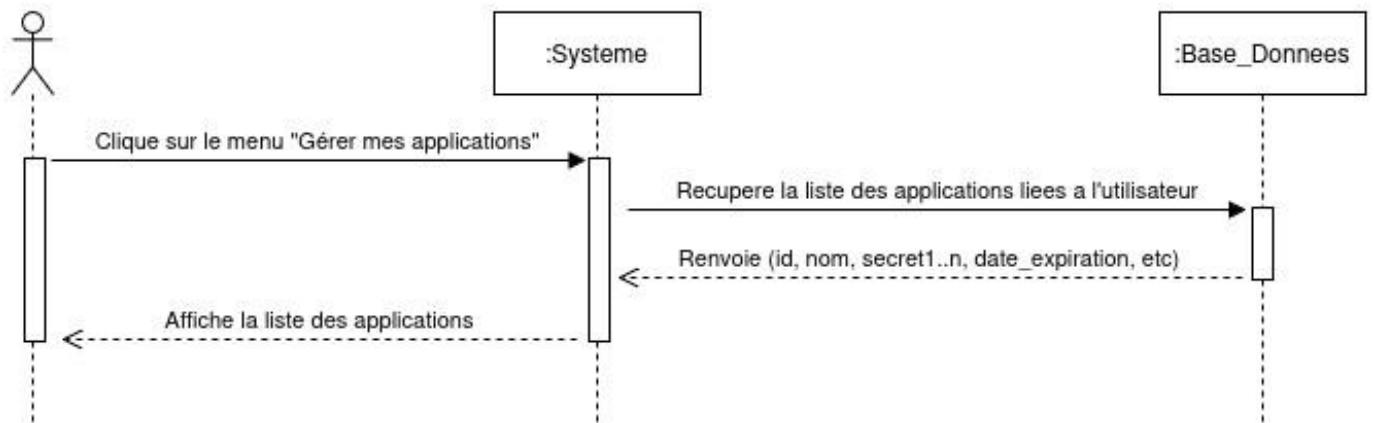
3. Modifier mon compte



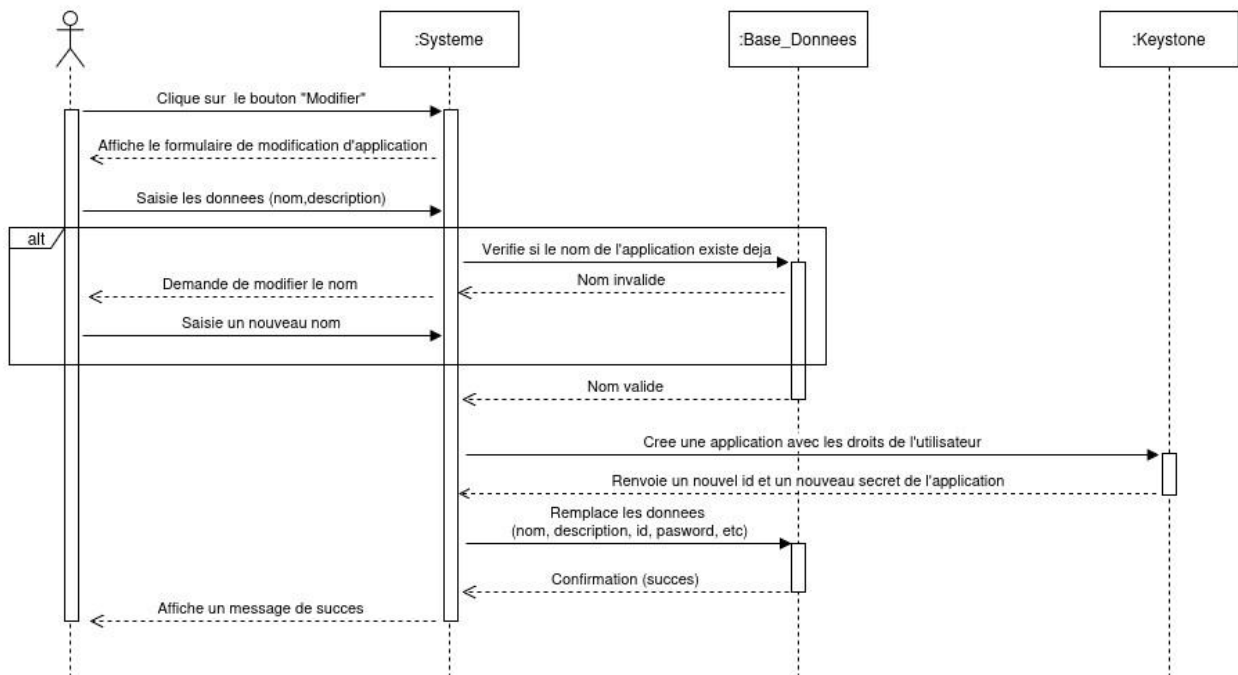
4. Ajouter une application



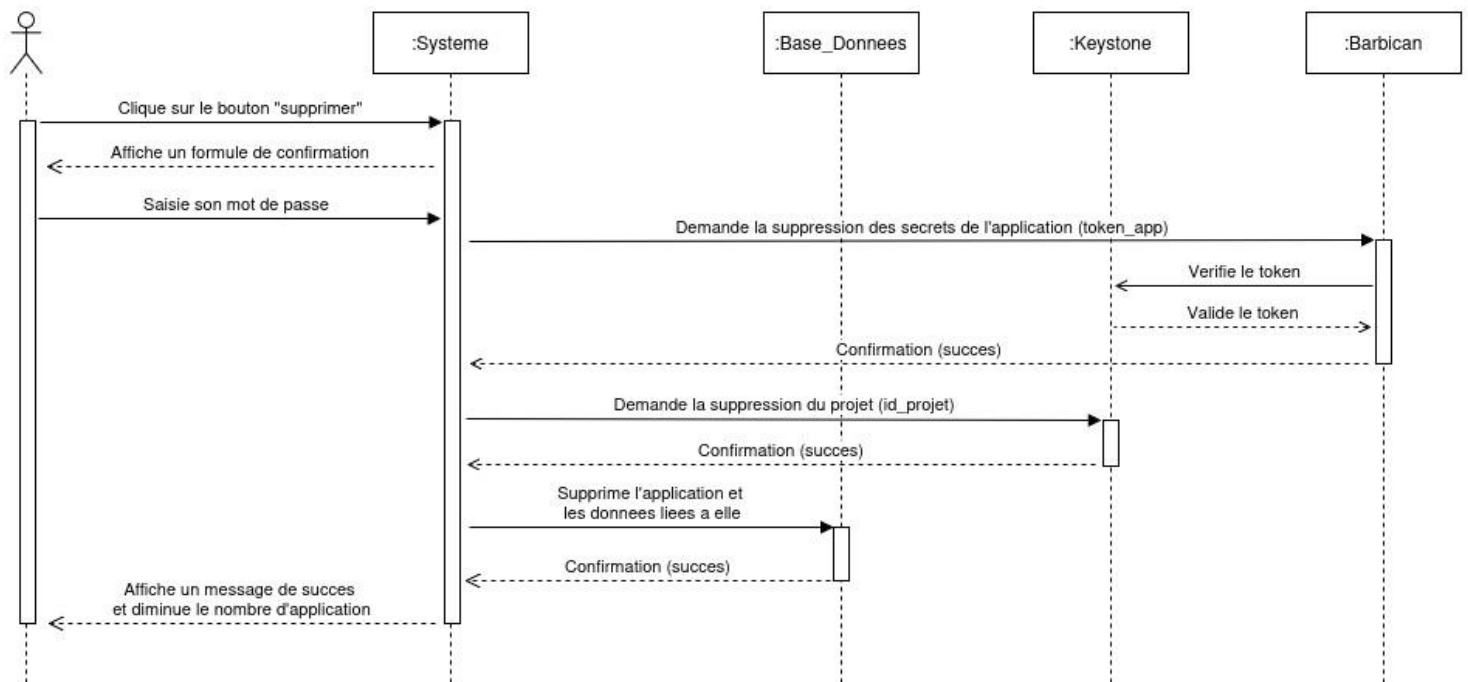
5. Visualiser les applications



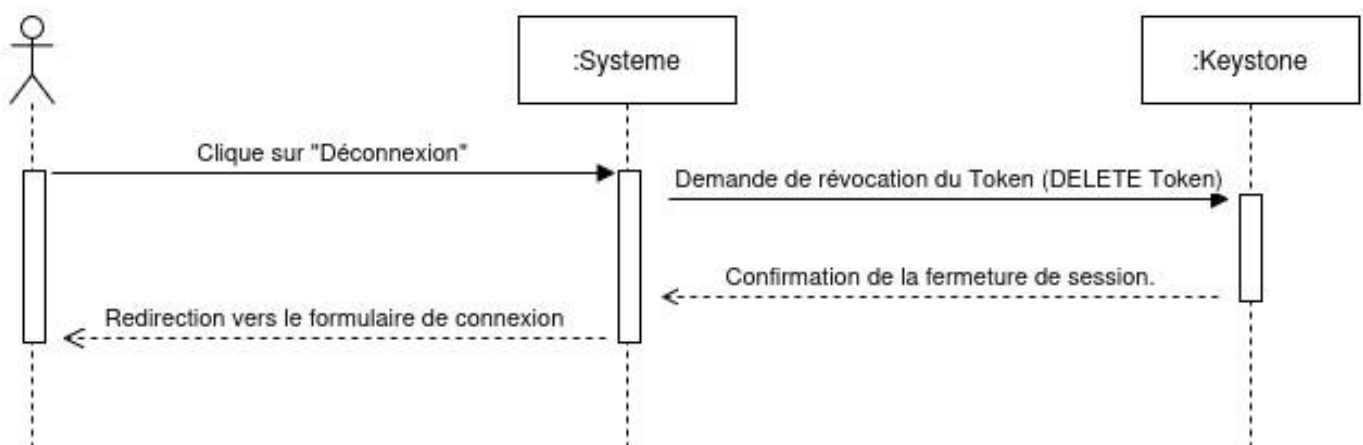
6. Modifier une application



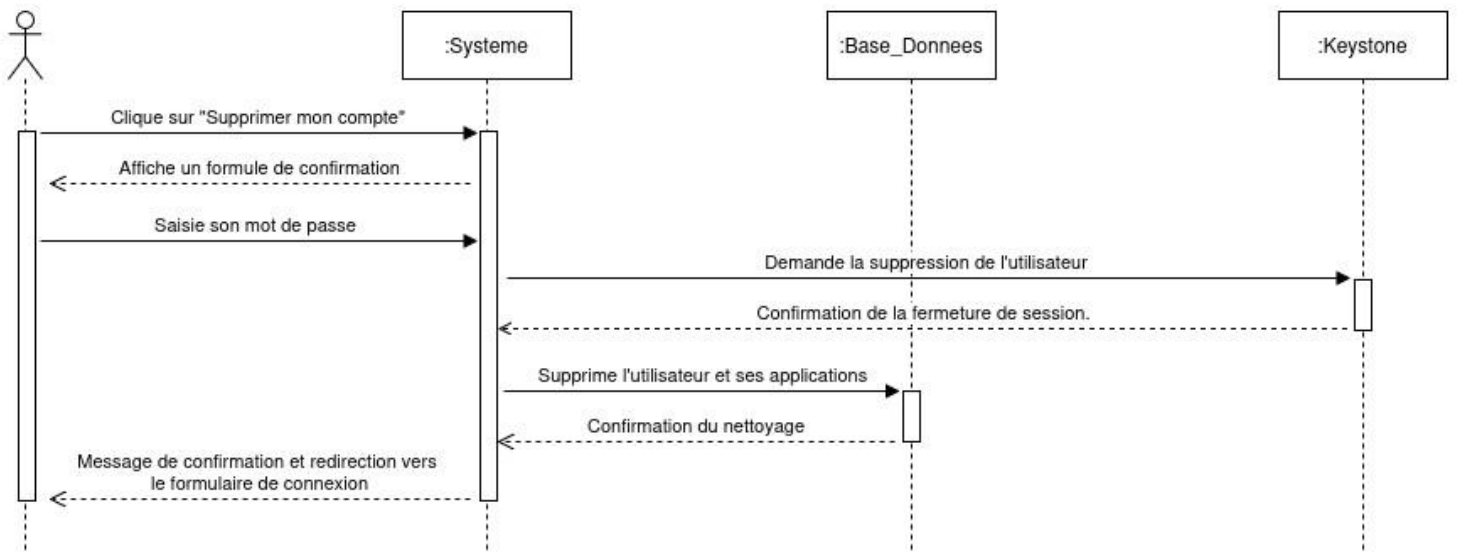
7. Supprimer une application



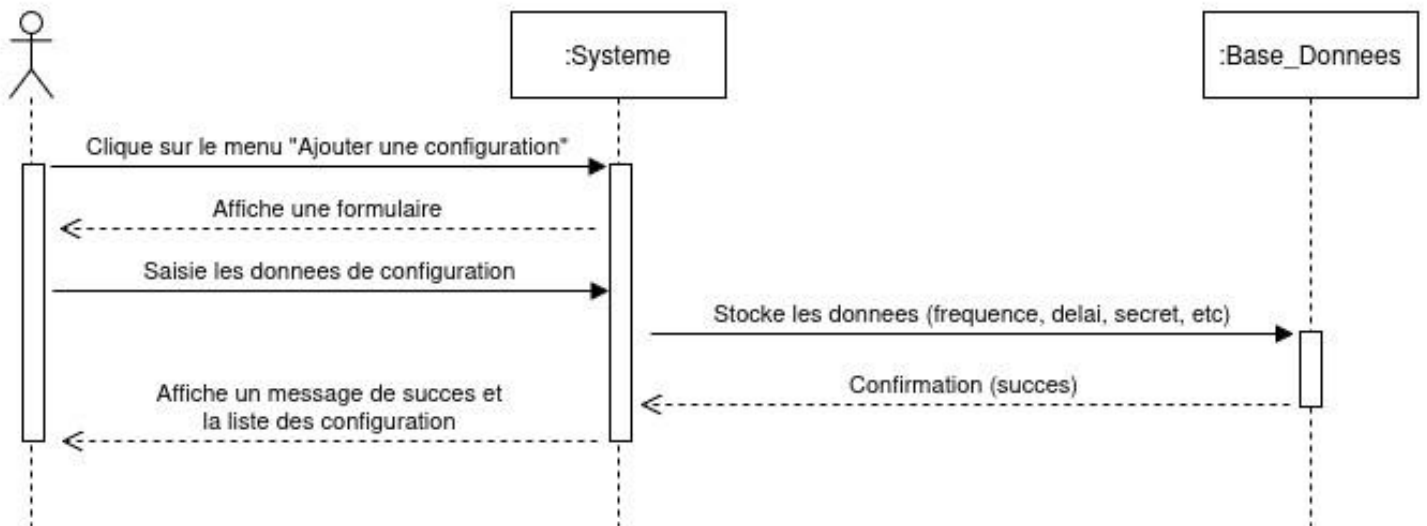
8. Se déconnecter



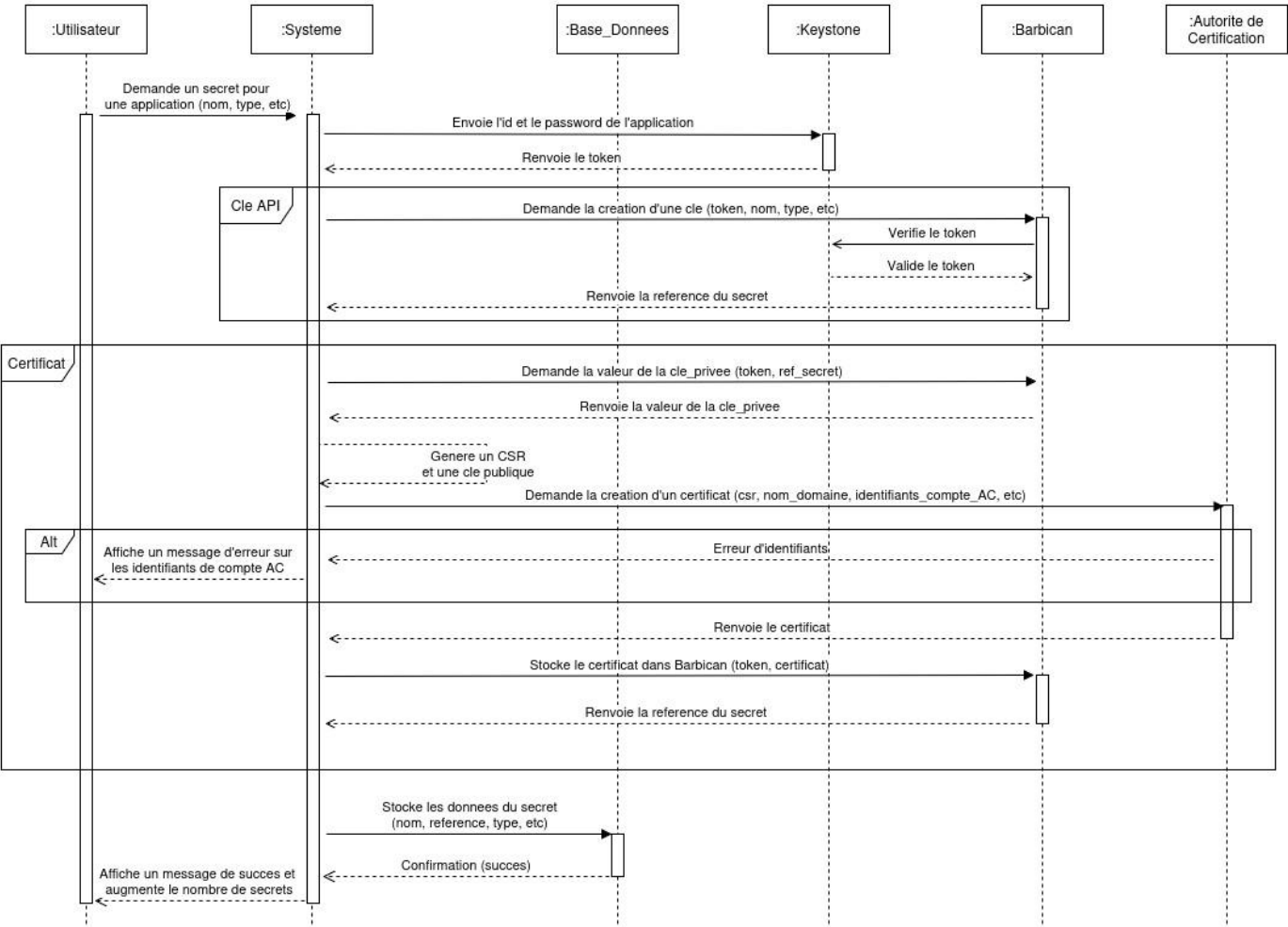
9. Supprimer son compte



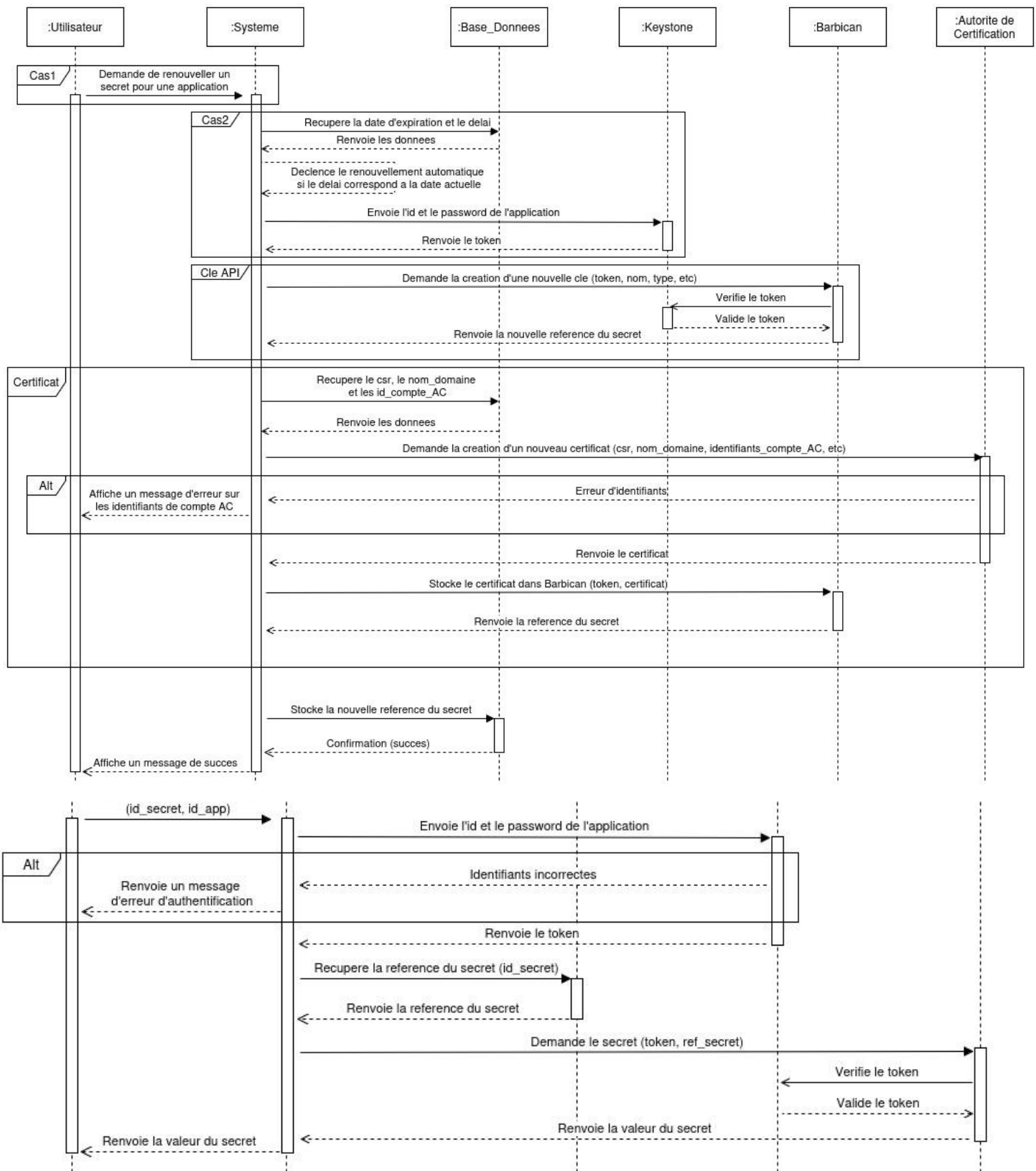
10. Configuration du renouvellement



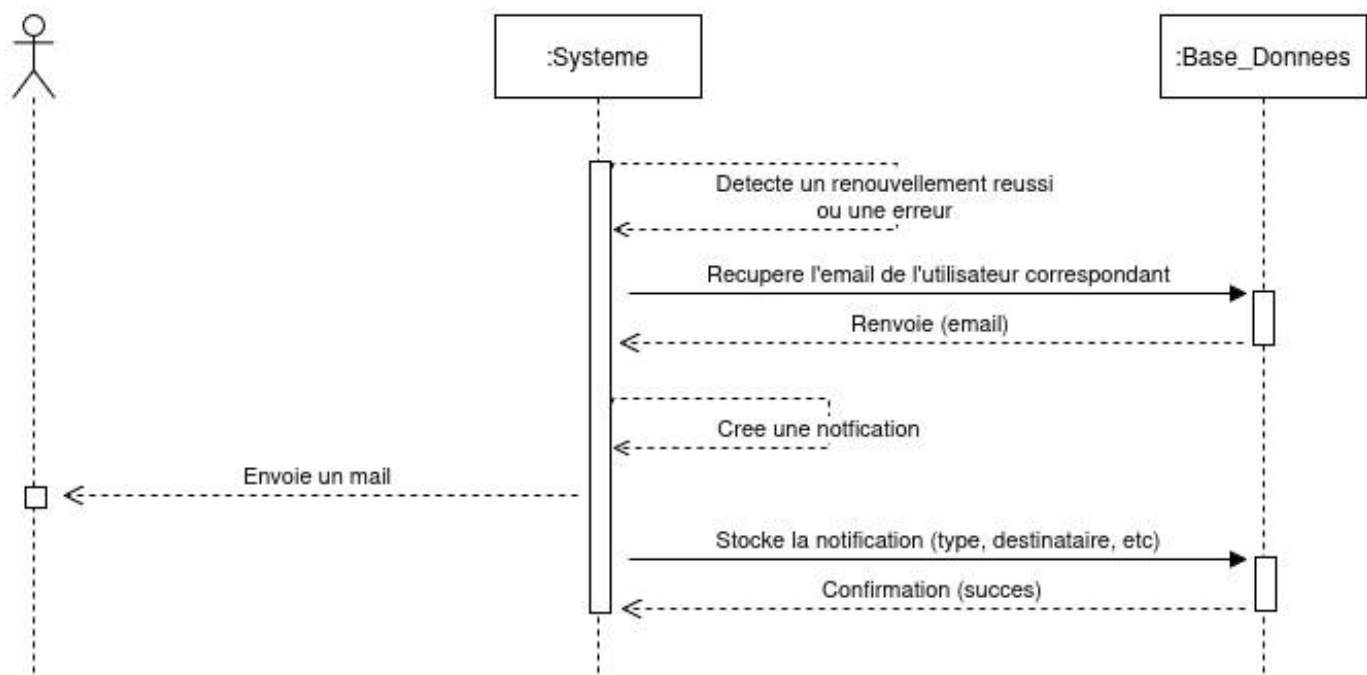
11. Ajouter un secret



12. Renouveler un secret



14. Notifier



IV. Découpage modulaire

1. Module de gestion des utilisateurs (Identity Management)

Ce module centralise le contrôle du cycle de vie des profils, depuis l'enrôlement des nouveaux administrateurs jusqu'à la révocation de leurs accès. Il assure la cohérence des informations personnelles et gère les états de session (connexion/déconnexion), garantissant que chaque action sur le système est imputable à un utilisateur identifié.

2. Module de gestion des applications (App Registry)

Ce module permet d'enregistrer et de répertorier l'ensemble des applications tierces autorisées à interagir avec Barbican. Il offre une visibilité centralisée sur le parc applicatif, permettant aux utilisateurs d'ajouter de nouvelles applications consommatrices de secrets et de consulter leurs caractéristiques spécifiques.

3. Module d'authentification (Security Gateway)

Sa mission est de valider l'identité de deux types d'acteurs : les utilisateurs humains (via interface) et les applications tierces (via API). En s'appuyant sur des protocoles de sécurité rigoureux, il délivre des jetons d'accès (tokens) temporaires qui définissent les droits et les limites de chaque entité au sein de notre système.

4. Module de gestion des secrets (Secret Lifecycle Manager)

Il orchestre toutes les opérations sensibles liées aux clés API et aux certificats. Au-delà du simple stockage, il gère l'automatisation des renouvellements, la révocation immédiate et la récupération sécurisée des données chiffrées. Il permet également de configurer les politiques de rotation pour assurer la pérennité des services dépendants.

V. Charte graphique

1. Couleurs

L'identité visuelle de notre solution repose sur un équilibre entre sobriété technologique et dynamisme. Le choix des couleurs a été dicté par la volonté d'offrir une interface lisible, sécurisante et moderne, facilitant la gestion de données critiques.

- **Le violet (couleur primaire)** : Il est utilisé pour les éléments d'action principaux (boutons, liens actifs) et pour l'identité de marque (logo). Il apporte la touche distinctive à l'interface ;
- **Le noir (couleur d'accent)** : Le noir est employé pour les structures fortes (titres principaux) ;

- **Le blanc (fond et clarté)** : Le blanc assure une lisibilité maximale et une interface aérée. Il permet de mettre en évidence les informations essentielles sans surcharge visuelle ;
- **Le gris (nuances de soutien)** : Le gris sert aux éléments secondaires (textes d'aide).

Couleur	Usage Principal
Violet	Actions, boutons, logo
Noir	Typographie
Gris	Texte d'aide, informations préremplies
Blanc	Menu de navigation, en-tête

2. Logo

L'identité visuelle de notre projet repose sur un logo hybride, fusionnant l'emblème officiel de Barbican avec notre dénomination propre, **BarbiLink**. Ce choix n'est pas seulement esthétique, il est hautement symbolique : le terme "Link" souligne explicitement la vocation de notre solution à servir de pont (ou liaison) entre le coffre-fort numérique Barbican et les applications tierces externes. Le logo incarne ainsi notre mission principale, qui est de créer une passerelle fluide, sécurisée et automatisée pour la gestion des secrets.



3. Interfaces graphiques

a) Les formulaires



Pseudonyme :

Mot de passe :

[Mot de passe oublié ?](#)

Connexion

Illustration 2: Formulaire de connexion

Veillez saisir votre mot de passe pour confirmer la suppression de votre compte

Confirmer

Illustration 3: Formulaire pour la suppression de compte



INSCRIPTION

Nom & prenom :

Pseudonyme :

Adresse Mail :

Mot de passe :

Confirmation du mot de passe :

Confirmer

Illustration 1: Formulaire d'inscription

Nouvelle application

Nom de l'application :

Description :

Nom de domaine :

Generer un secret →

Confirmer

Illustration 4: Formulaire d'ajout d'application

Nouveau secret

Nom du secret :

Type de secret :

☐ Cle API ☐ Certificat SSL/TLS

Algorithme de chiffrement :

Date d'expiration : __/__/__

Valeur du secret (optionnel) :

Ajouter

Illustration 5: Formulaire d'ajout d'un secret

Configuration du renouvellement automatique

Frequence de rotation : 

Fichier CSR :

Delai d'anticipation : Jour(s)

Application cible :



Secret cible :



Enregistrer

Illustration 6: Formulaire de configuration du renouvellement automatique des secrets

b) Les pages

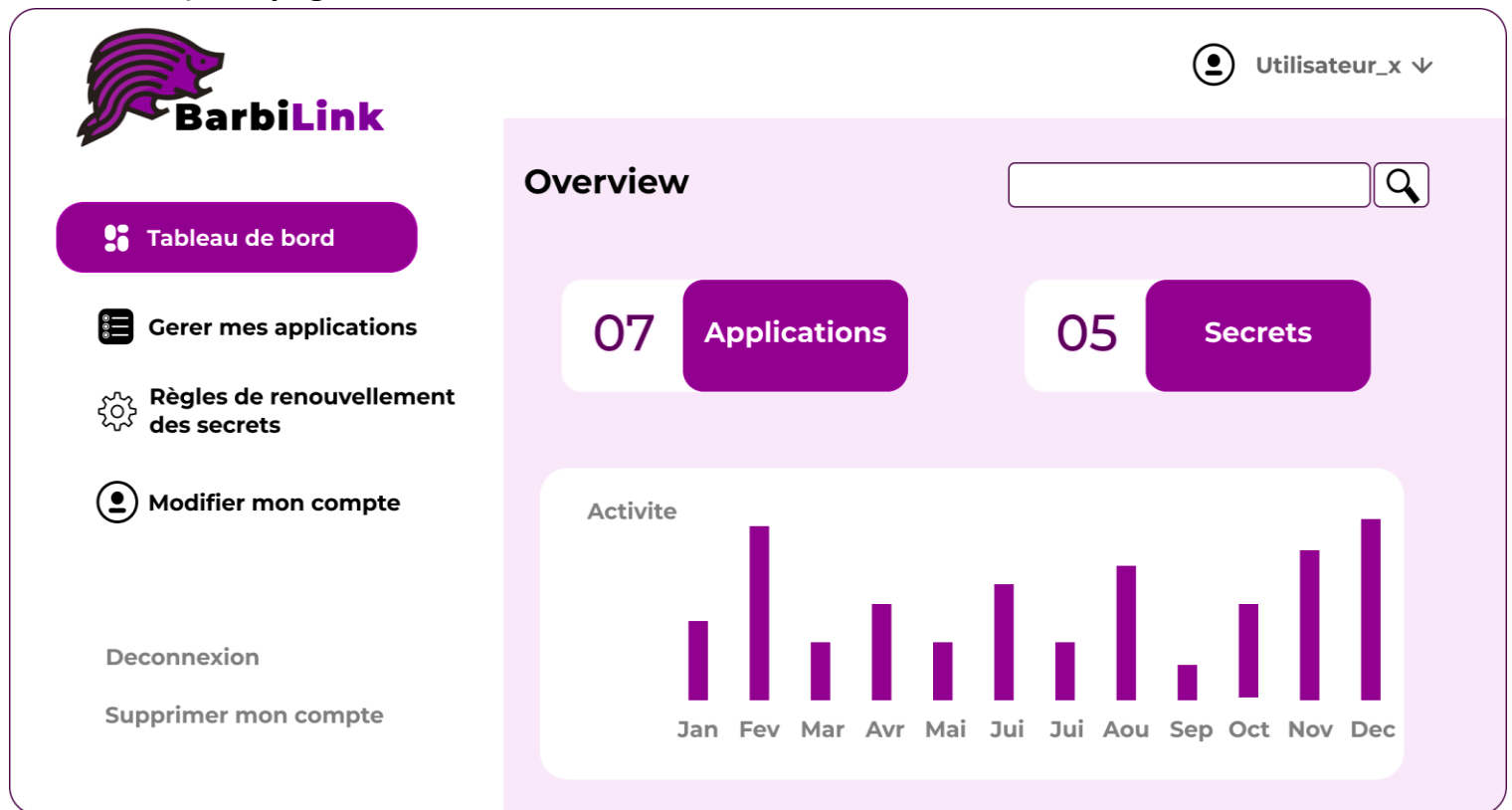


Illustration 7: Page d'accueil après connexion

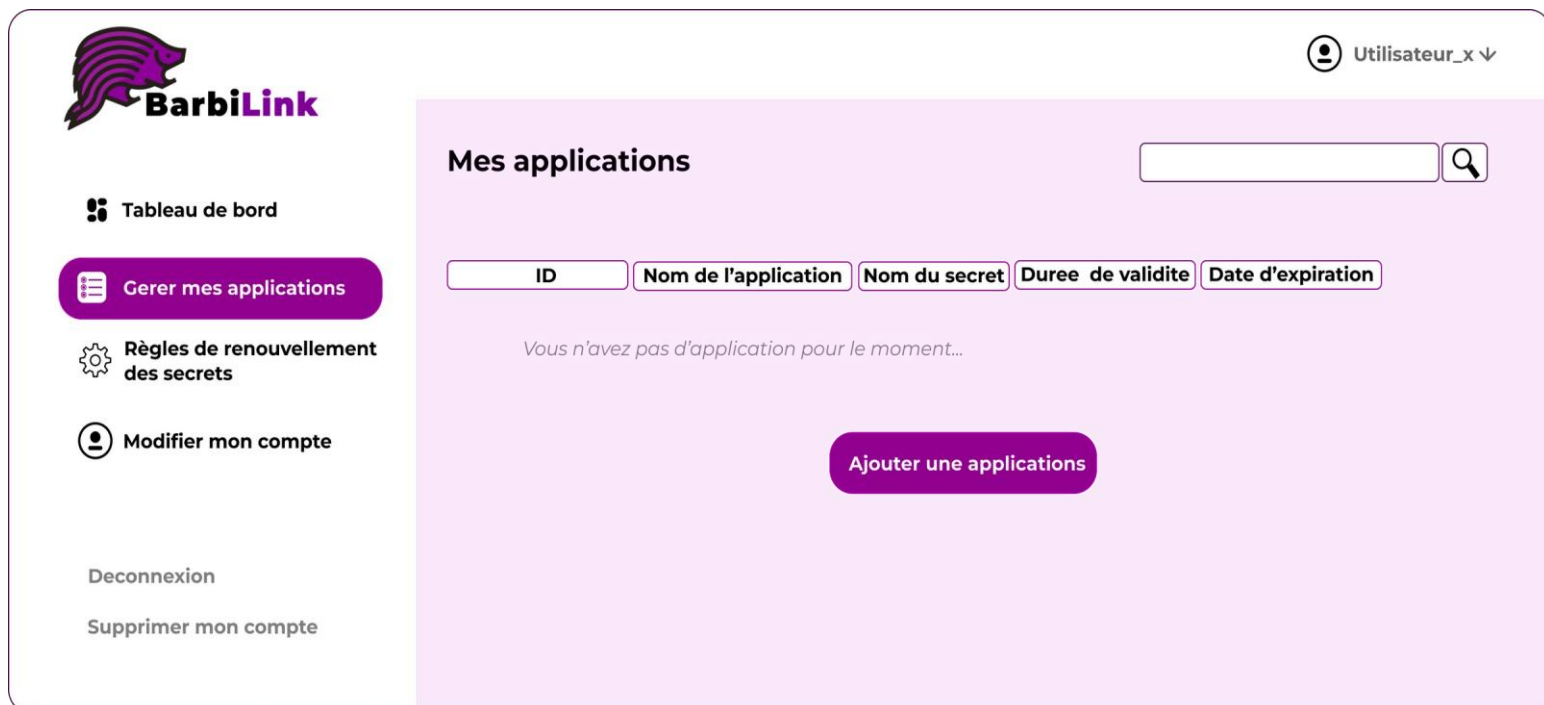


Illustration 8: Page affichant la section applications lorsqu'il n'y a pas d'applications



 Utilisateur_x ▼

 Tableau de bord

 **Gérer mes applications**

 Règles de renouvellement des secrets

 Modifier mon compte

Deconnexion
Supprimer mon compte

Mes applications

ID	Nom de l'application	Nom du secret	Durée de validité	Date d'expiration	
fg38ds-3fedo-38hdw-3478hd	PayCash	Cle_payCash	01 mois	15 Mars 2026	 
fghe0-ehdkala-ffedew5-dnsde49	MeteoCam				 

Ajouter une applications

Illustration 9: Page affichant la section applications lorsqu'il a des applications



 Utilisateur_x ▼

 Tableau de bord

 Gérer mes applications

 **Règles de renouvellement des secrets**

 Modifier mon compte

Deconnexion
Supprimer mon compte

Configuration du renouvellement

Nom de l'application	Nom du secret	Fréquence de rotation	Fichier CSR	Délai d'anticipation
Vous n'avez pas de configuration pour le moment...				

Nouvelle configuration

Illustration 10: Page affichant la section de configuration du renouvellement sans configuration

 Tableau de bord Gérer mes applications Règles de renouvellement des secrets Modifier mon compte

Déconnexion

Supprimer mon compte

Configuration du renouvellement

Nom de l'application	Nom du secret	Fréquence de rotation	Fichier CSR	Délai d'anticipation
----------------------	---------------	-----------------------	-------------	----------------------

PayCash

Cle_payCash

03 semaines

/

15 jours avant

Nouvelle configuration

Illustration 11: Page affichant la section de configuration du renouvellement avec une configuration

 Tableau de bord Gérer mes applications Règles de renouvellement des secrets Modifier mon compte

Déconnexion

Supprimer mon compte

Mon compte

Modifier mon identité

Nom & prenom : Pseudonyme : Adresse mail : Modifier

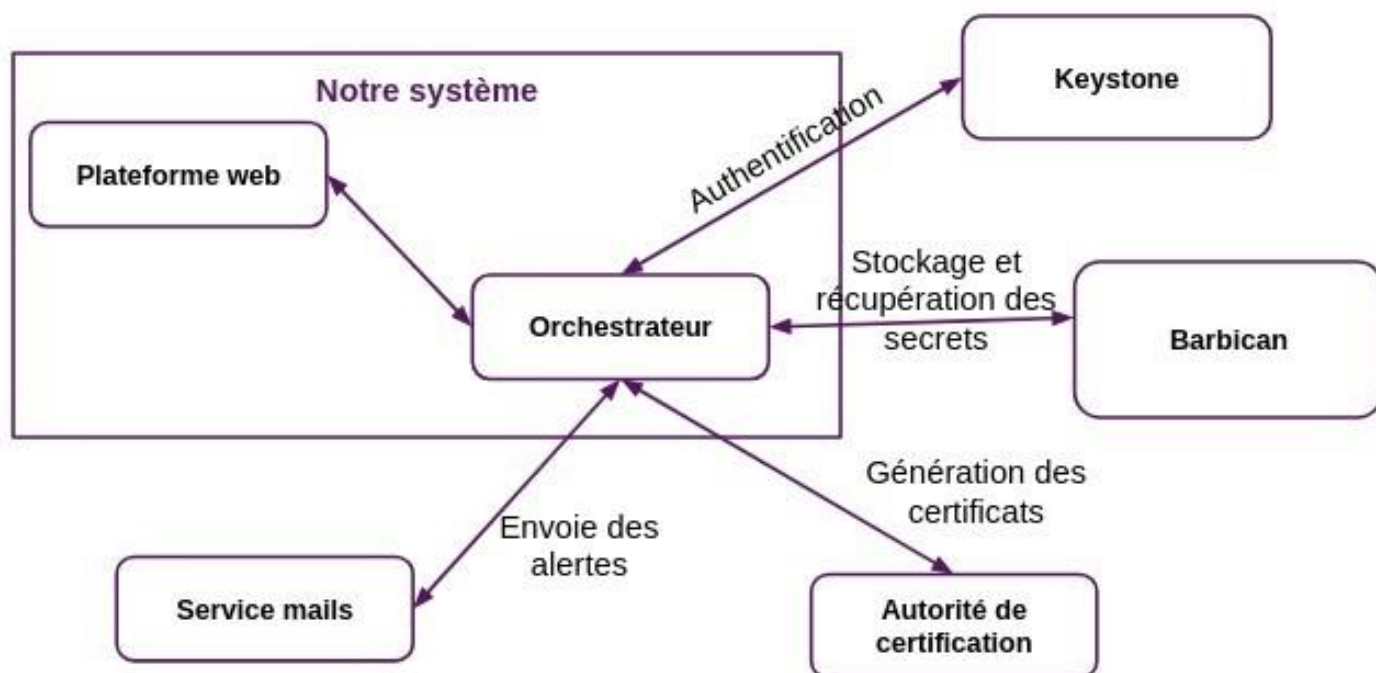
Modifier mon mot de passe

Ancien mot de passe : Nouveau mot de passe : Confirmer le nouveau mot de passe : Modifier

Illustration 12: Page affichant les détails du compte pour modification

VI. Architecture macro de la solution BarbiLink

Voici comment BarbiLink s'intègre dans l'infrastructure de sécurité :



Cette architecture, que l'on peut qualifier de schéma fonctionnel de la solution BarbiLink, présente une organisation modulaire centrée sur l'automatisation et l'interconnexion. Elle met en lumière le rôle pivot de votre système comme passerelle entre l'utilisateur et l'écosystème OpenStack.

1. Le Cœur du système : BarbiLink

Notre système est composé de deux briques internes majeures :

- La plateforme web : Elle constitue l'interface utilisateur (UI). C'est le point d'entrée pour les utilisateurs qui souhaitent configurer les applications et superviser les secrets ;
- L'orchestrateur : C'est le moteur "intelligent" de la solution. Il reçoit les requêtes de la plateforme web et coordonne l'ensemble des actions avec les services tiers. Il assure la logique métier et le routage des données.

2. Les Interactions externes (Services tiers)

L'orchestrateur communique avec quatre entités externes pour accomplir ses missions :

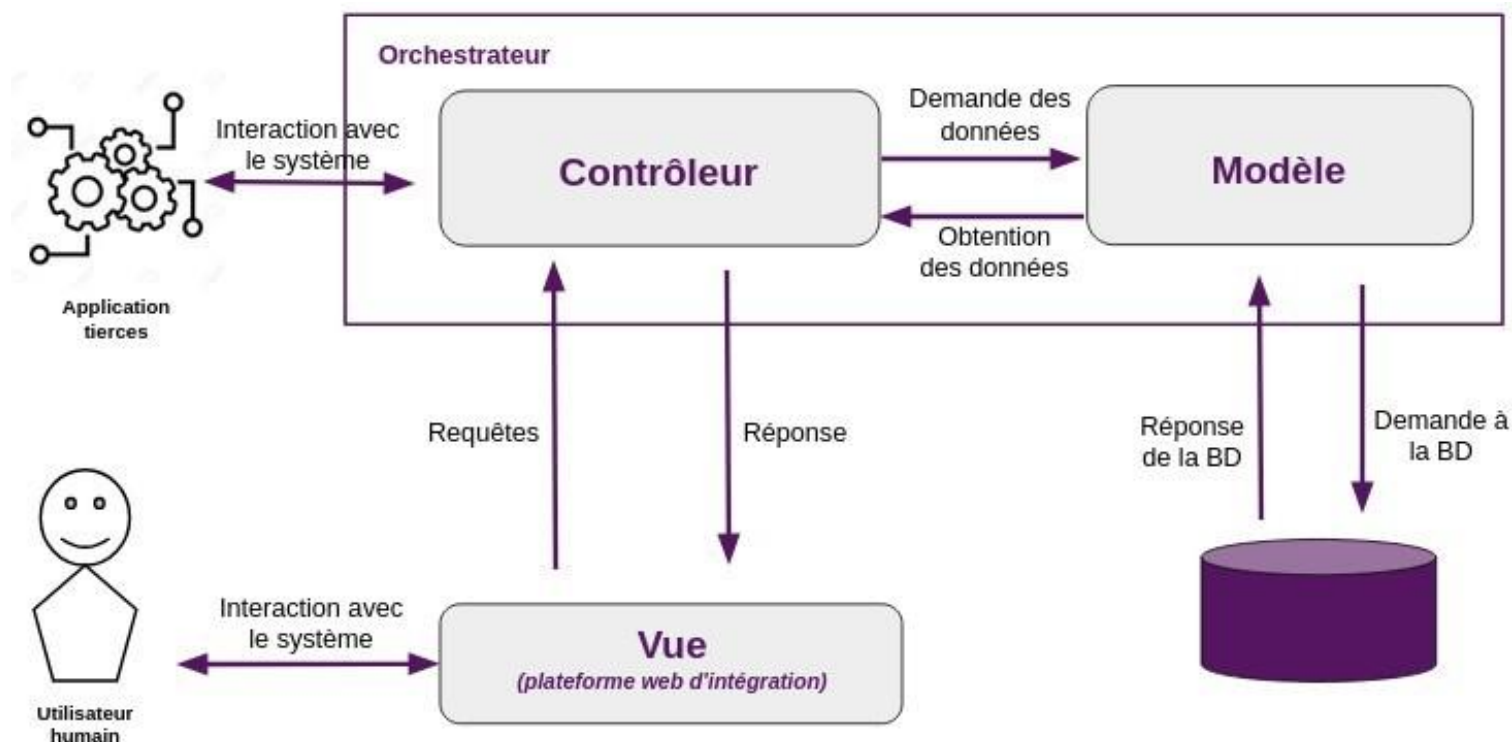
- Keystone (Authentification) : L'orchestrateur interroge Keystone pour valider les identités et obtenir les jetons d'accès nécessaires aux opérations sécurisées ;
- Barbican (Gestion des secrets) : L'orchestrateur y stocke et récupère les clés et certificats de manière sécurisée ;
- Autorité de certification (PKI) : Pour l'objectif d'automatisation, l'orchestrateur pilote la génération et le renouvellement des certificats SSL/TLS auprès de cette entité ;
- Service mails (Notifications) : En cas d'alerte (renouvellement effectué, erreur), l'orchestrateur déclenche l'envoi automatique de courriels pour informer les utilisateurs.

3. La valeur ajoutée : Le flux de liaison

Le schéma illustre parfaitement le concept de "pont". L'orchestrateur masque la complexité technique de Barbican et Keystone derrière une interface simplifiée, tout en ajoutant des fonctionnalités qui n'existent pas nativement, comme la génération automatisée et l'envoi d'alertes.

VII. Patron de conception de la solution

Pour garantir la modularité et la maintenance du code, nous avons adopté une structure interne de type MVC, détaillée ci-dessous :



1. La vue (Interface Utilisateur)

Il s'agit de votre plateforme web d'intégration. C'est l'interface avec laquelle l'utilisateur humain interagit. Elle affiche les informations (secrets, liste des applications) et envoie les requêtes vers le contrôleur dès qu'une action est effectuée.

2. L'orchestrateur (Le cœur logique)

C'est ici que se trouve l'intelligence de notre application, divisée en deux parties :

- Le contrôleur : Il reçoit les requêtes de la vue ou des applications tierces. Il demande les données nécessaires au modèle, reçoit la réponse, puis renvoie la réponse finale à l'utilisateur ou au système externe ;

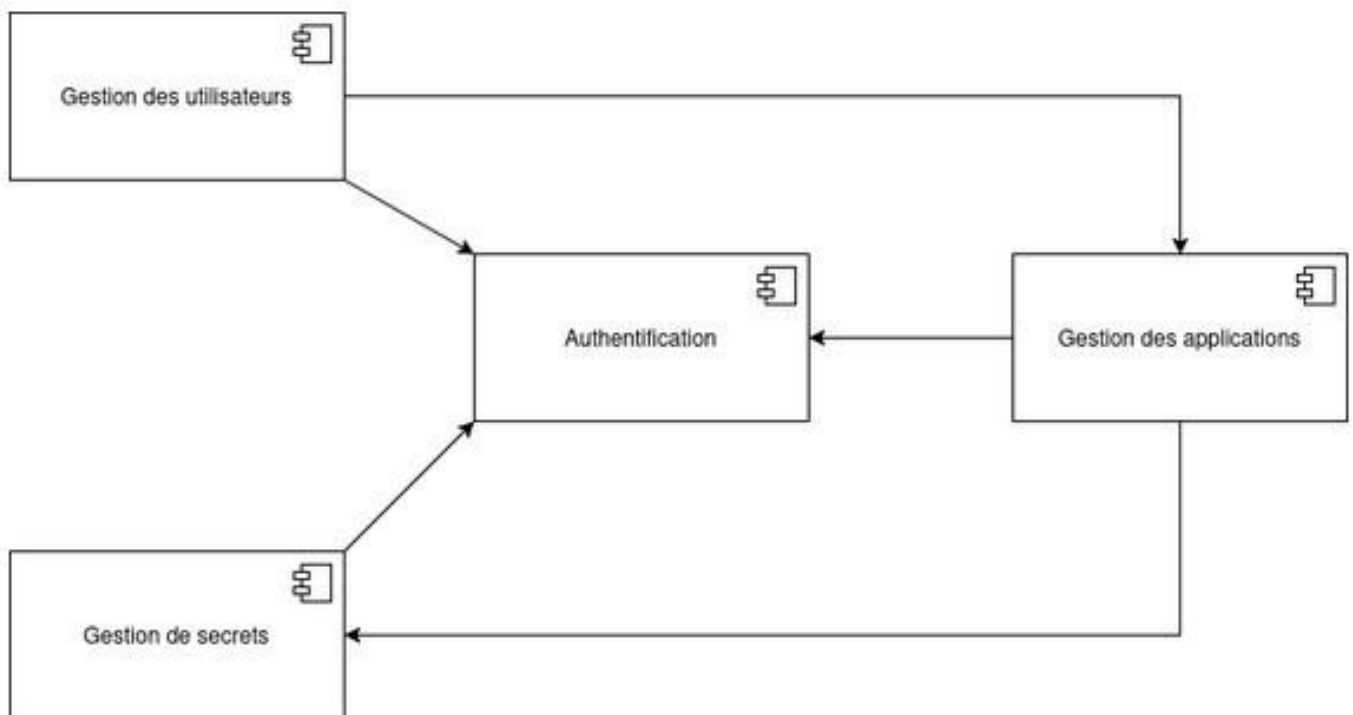
- Le modèle : Il gère la logique des données. C'est lui qui sait comment formater une demande pour la base de données et comment interpréter les informations reçues pour les transmettre proprement au contrôleur.

3. La base de données (Stockage)

Représentée par le cylindre violet, elle stocke de manière persistante toutes les informations de notre système (identifiants des applications, profils utilisateurs). Le modèle y effectue des demandes et reçoit des réponses.

VIII. Diagramme des composants

Voici comment les composants de notre système interagissent :



1. L'authentification

Le module Authentification occupe une position centrale. Les flèches pointant vers lui depuis la *Gestion des utilisateurs*, la *Gestion de secrets* et la *Gestion des applications* indiquent une dépendance critique :

- Aucune action ne peut être effectuée dans les autres modules sans passer par une vérification d'identité ;
- Ce module centralise la distribution des jetons (tokens) nécessaires pour autoriser les appels API ou les accès aux interfaces.

2. Le flux de gestion des applications

Le module Gestion des applications agit comme un intermédiaire stratégique :

- Lien avec les utilisateurs : La flèche venant de la *Gestion des utilisateurs* vers la *Gestion des applications* montre que ce sont les utilisateurs (humains) qui déclarent et supervisent les applications tierces ;
- Lien avec les secrets : La flèche pointant vers la *Gestion de secrets* indique que les secrets (clés API, certificats) sont directement rattachés à des applications spécifiques. On ne crée pas un secret "dans le vide", on le crée pour une application enregistrée.

3. Le cycle de vie des secrets

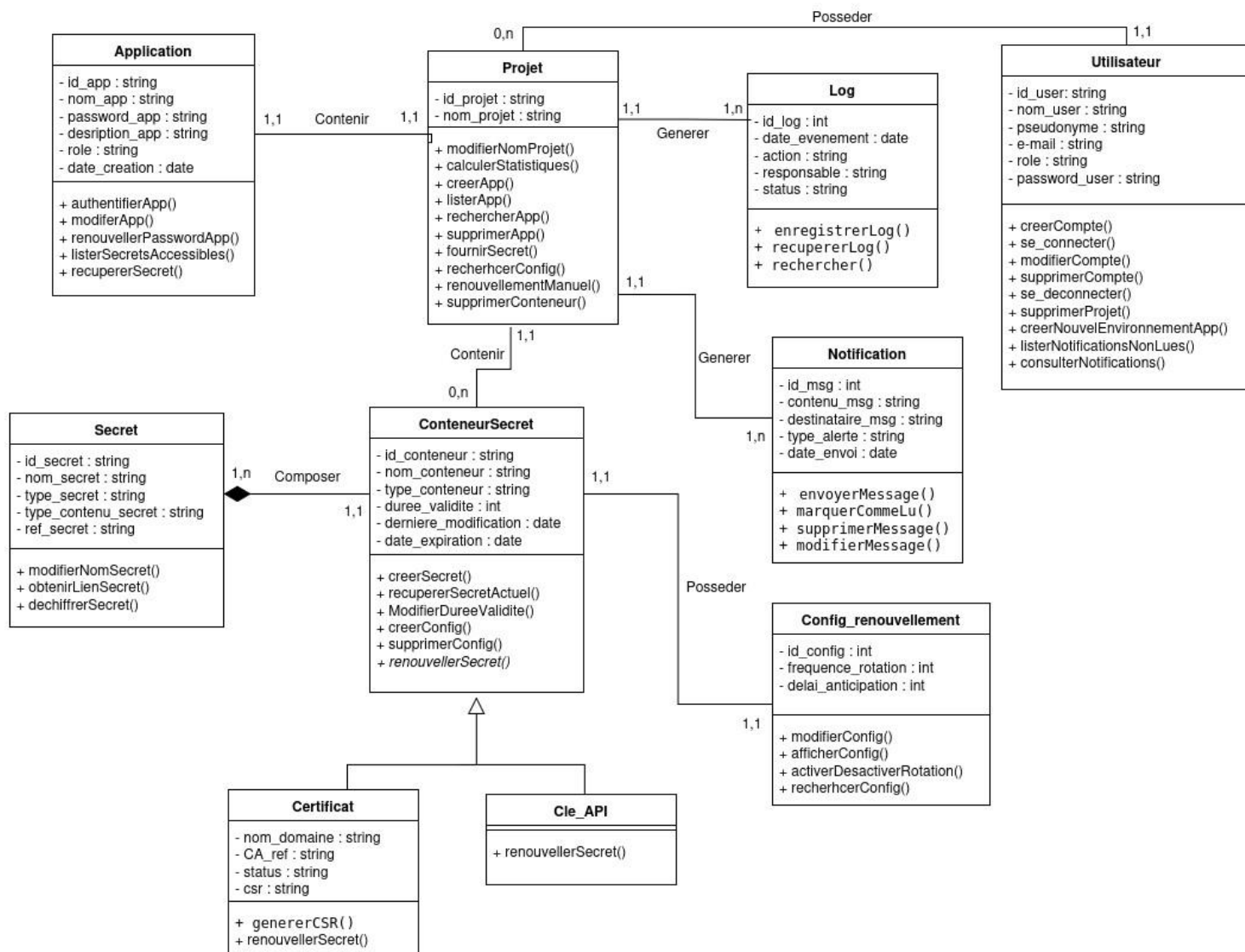
Le module *Gestion de secrets* est le point d'aboutissement technique. Il reçoit les ordres de configuration depuis le module *Gestion des applications* et valide chaque requête auprès du module d'authentification pour garantir qu'aucune donnée sensible ne fuite.

4. Détail des flux inter-applicatifs

Émetteur (Source)	Récepteur (Destination)	Donnée transmise	Protocole / Méthode	Finalité
Gestion des Utilisateurs	Authentification	Identifiants (Login/Pass)	HTTPS / POST	Vérifier l'identité de l'admin.
Authentification	Gestion Secrets	Jeton d'accès (JWT)	Header Auth	Autoriser l'ajout/lecture d'un secret.
Gestion des applications	Gestion Secrets	ID_App + Type secret	Interne / API	Lier un secret à une application tierce.
Gestion de secrets	Barbican (Externe)	Payload chiffré	REST API	Stocker le secret dans le coffre-fort.
Système	Serveur Mail	Email + Message d'alerte	SMTP	Notifier de l'expiration d'un certificat.
Authentification	Keystone (Externe)	Identifiants	REST (HTTPS)	Valider l'identité via l'annuaire OpenStack.
Keystone (Externe)	Authentification	Token de session	JSON / Response	Autoriser l'accès aux services BarbiLink.
Gestion de secrets	Barbican (Externe)	Secret chiffré + Métadonnées	Barbican API	Stocker ou récupérer une clé/certificat.
Gestion de secrets	PKI (Autorité de certification)	CSR (Demande de signature)	ACME / SCEP	Générer ou renouveler un certificat SSL.
PKI (Autorité de certification)	Gestion de secrets	Certificat signé (.crt)	HTTPS	Récupérer le nouveau certificat valide.
Système	Service Mail (Externe)	Erreur / notification de renouvellement	SMTP / API	Alerter l'utilisateur du renouvellement.

Application tierce (Externe)	Authentification	ID_App + Type secret	REST / POST	Obtenir un jeton pour consommer un secret.
---	-------------------------	-------------------------	-------------	---

IX. Diagramme de classe



1. Classe Application

- **authentifierApp(id_app: string): string** (renvoie le token keystone)
- **modifierApp(id_app: string, nouveauNom: string, description: string): void**
- **renouvelerPasswordApp(id_app: string): string** (retourne un nouveau secret de l'application)
- **listerSecretsAccessibles(id_projet: string): List<Secret>**
- **recupererSecret(id_secret: string, token : string): Secret**

2. Classe Projet

- **modifierNomProjet(id_projet: string, nouveauNom: string): void**
- **calculerStatistiques(): <int, int>** (renvoie le nombre de secrets et le nombre d'applications)
- **creerApp(nom_app: string, description_app: string, role: string): <string, string>** (retourne l'id de l'application et le secret de l'application)
- **listerApp(id_projet: string): List<Application>**
- **rechercherApp(param: string): Application**
- **supprimerApp(id_app: string): void**
- **fournirSecret(id_secret: string, app_token: string): Secret**
- **rechercherConfig(param: string): Config_renouvellement**
- **renouvellementManuel(id_conteneur: string): void**
- **supprimerConteneur(id_conteneur: string): void**

3. Classe Log

- **EnregistrerLog(id_projet: string, action: string, responsable: string, status: string): void**
- **recupererLog(id_projet: string): List<Log>**
- **rechercher(param : string): List<Log>**

4. Classe Utilisateur

- **creerCompte(nom_user: string, pseudo: string, email: string, password_user: string, role: string): string** (retourne l'id de l'utilisateur)
- **se_connecter(pseudo: string, mdp: string): string** (retourne le token)
- **modifierCompte(id_user: string, nom_user: string = null, pseudo: string = null, email: string = null, password_user: string = null): void**
- **supprimerCompte(id_user: string): void** • **se_deconnecter(): void**
- **supprimerProjet(id_projet: string): void**
- **creerNouvelEnvironnementApp(nom_app: string, role: string, description_app : string, nom_secret: string, duree_validite : int, type_secret): <string, string, string, string, string, string>** (retourne les id du projet, de l'application, du secret et du conteneur, le secret de l'application, la reference du secret). Cette fonction doit utiliser le nom_app pour definir le nom_projet, le nom_secret pour definir le nom_conteneur, le type_secret pour definir le type_conteneur.
- **listerNotificationsNonLues(id_projet: string): List<Notification>**
- **consulterNotifications(id_projet: string): List<Notification>**

5. Classe Config_renouvellement

- **modifierConfig(id_config: int): int**
- **rechercherConfig(param: string): List<Config_renouvellement>**
- **activerDesactiverRotation(id_config: int): void**

- **afficherConfig(id_config: string): void**

6. Classe Secret

- **ModifierNomSecret(id_secret: string, nouveauNom: string): void**
- **obtenirLienSecret(id_secret: string): string**
- **dechiffrerSecret(id_secret: string, type_contenu_secret: string): string**

7. Classe Notification

- **envoyerMessage(contenu_msg: string, destinaire_msg: string, type_alerte: string): void**
- **marquerCommeLu(id_msg: int): void**
- **supprimerMessage(id_msg: int): void**
- **modifierMessage(id_msg: int, contenu_msg: string = null, type_alerte: string = null): void**

8. Classe ConteneurSecret

- **creerSecret(id_conteneur : string, nom_secret : string, type_secret : string, type_contenu_secret : string) : <string, string>** (retourne l'id et la reference du secret)
- **recupererSecretActuel(id_conteneur : string, type_contenu_secret : string) : string** (retourne la reference du secret)
- **ModifierDureeValidite(id_conteneur: string, nouvelle_duree: int): void**
- **creerConfig(id_conteneur: string, frequence: int, delai: int): void**
- **SupprimerConfig(id_config: int): void**
- **renouvelerSecret(): string**

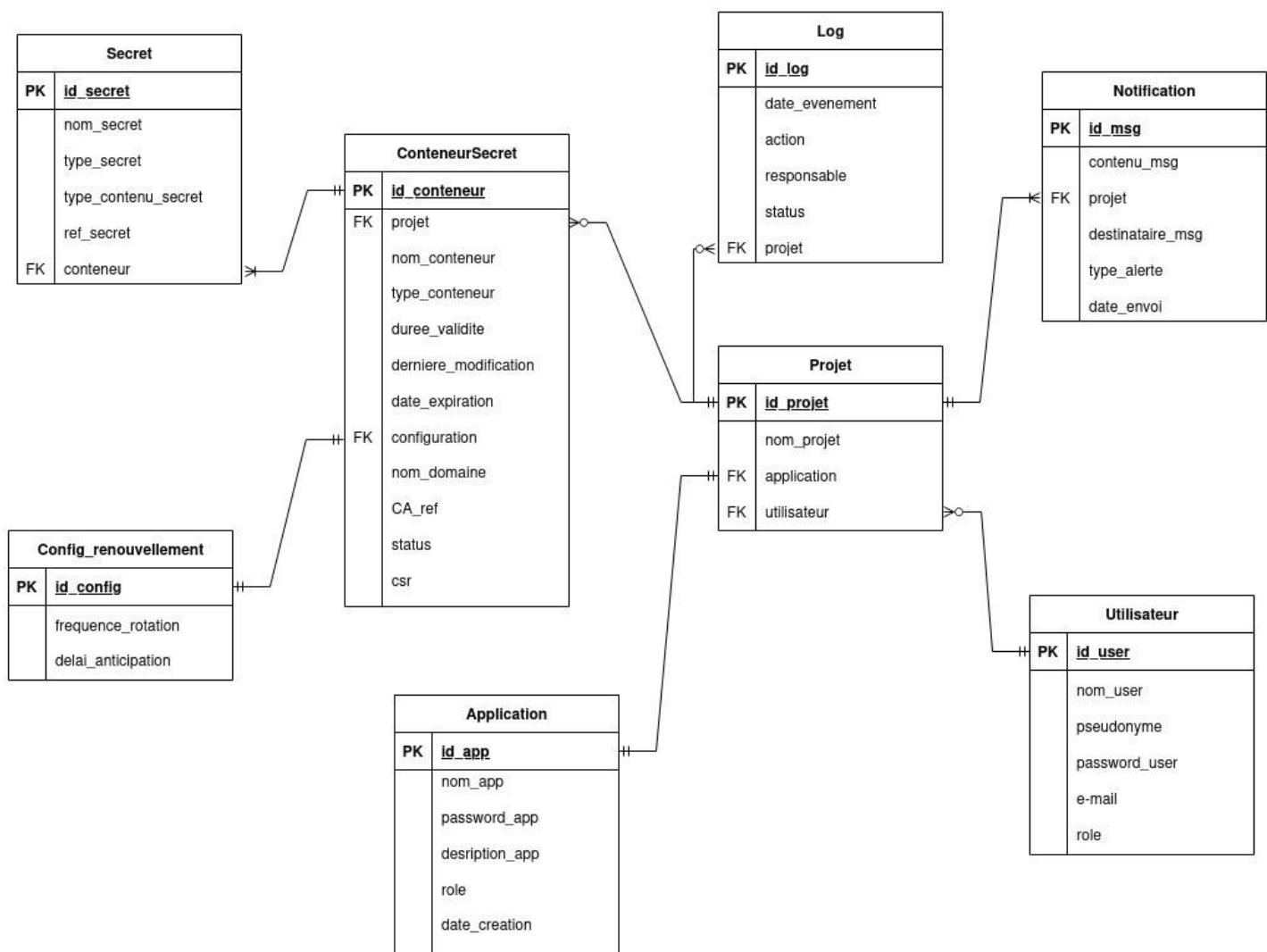
9. Classe Cle_API

- **RenouvelerSecret(id_conteneur : string) : <string, string>** (retourne l'id et la reference de la nouvelle clé API)

10. Classe Certificat

- **genererCSR(id_conteneur: string, nom_domaine: string): string** (retourne le csr)
- **renouvelerSecret(id_conteneur : string, CA_ref: string, csr: string): <string, string>** (retourne l'id et la reference de la nouvelle clé API)

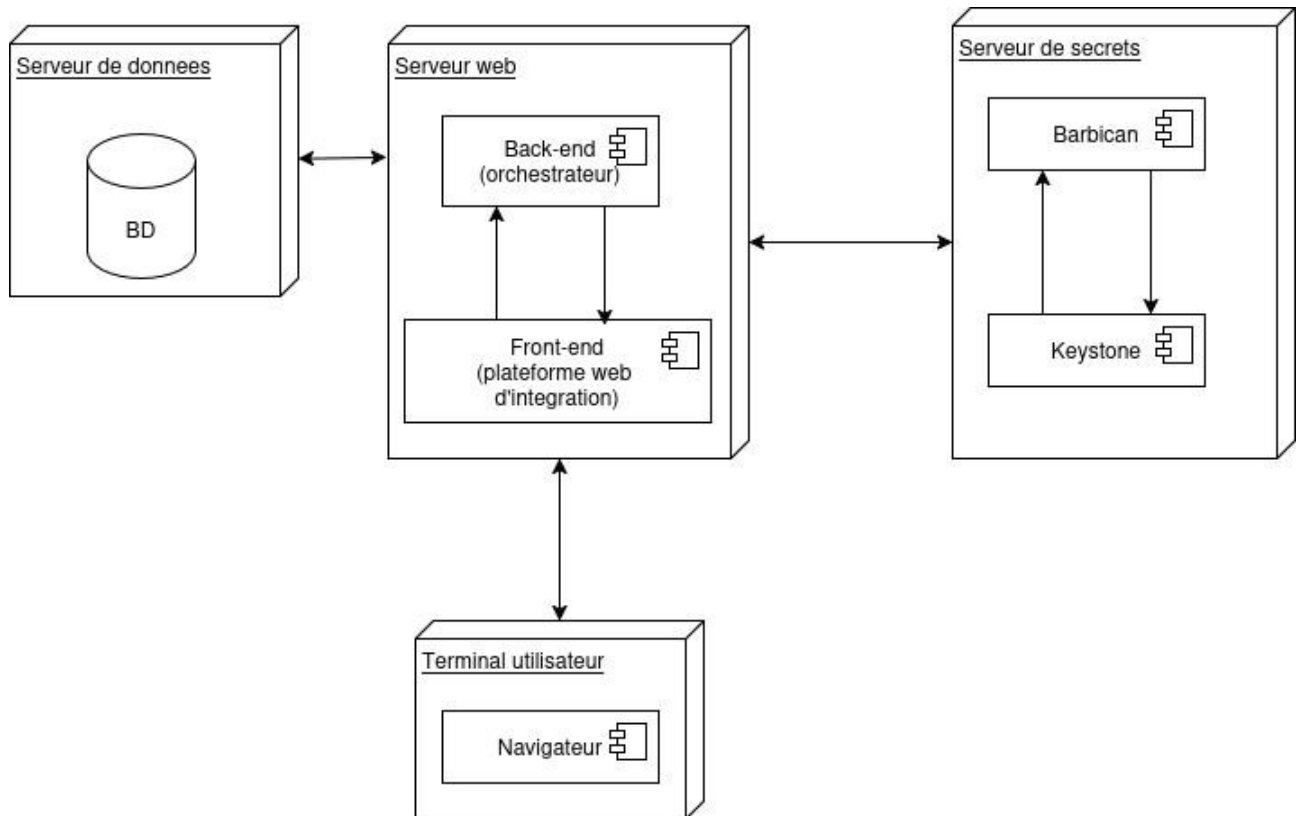
X. Schéma de la base de données



Points dont il faut tenir compte :

- **Gestion des suppressions (Cascade)** : Si un projet est supprimé, toutes les applications et tous les secrets associés doivent disparaître pour éviter des données "orphelines" en base ;
- **Stockage des données** : Toutes les données doivent être chiffrées au repos dans la base de données ;
- **Champs nullables** : Les colonnes spécifiques aux certificats (nom_domaine, CA_ref, csr, status) doivent impérativement accepter les valeurs NULL ;
- **Unicité de la configuration** : Dans ConteneurSecret, la colonne id_config (ou configuration) doit avoir une contrainte UNIQUE ;
- **Indexation** : Il faut indexer toutes les clés étrangères (FK), les colonnes type_conteneur, email, date_expiration et date_envoi pour accélérer les jointures, les filtres et les tris.

XI. Diagramme de déploiement



1. Terminal utilisateur

C'est le point d'entrée. L'utilisateur interagit avec le système via un **navigateur web**. C'est ici que les requêtes sont émises vers l'infrastructure.

2. Serveur web (Le cœur du système)

Il contient deux composants majeurs qui gèrent la logique métier :

- **Front-end** : La plateforme web d'intégration qui reçoit les demandes du navigateur.
- **Back-end (orchestrateur)** : Il reçoit les instructions du Front-end et joue le rôle de chef d'orchestre. C'est lui qui va discuter avec la base de données pour le stockage et avec le serveur de secrets pour la sécurité.

3. Serveur de données

Ce bloc contient la **BD (Base de Données)**. C'est là que sont stockées les informations que nous avons modélisées précédemment (utilisateurs, projets, métadonnées des secrets). Le Back-end interroge ce serveur pour lire ou enregistrer l'état du système.

4. Serveur de secrets (L'infrastructure de sécurité)

Il s'agit de la partie inspirée d'OpenStack, regroupant deux services critiques :

- **Keystone** : Gère l'authentification et les autorisations (Identity Service).
- **Barbican** : Gère le cycle de vie des secrets (Key Manager Service).

- **Communication** : Ces deux services échangent entre eux pour valider que l'utilisateur qui demande un secret via Barbican possède bien les droits dans Keystone.